

Vertex AI：訓練及提供自訂模型

來源：https://codelabs.developers.google.com/vertex_custom_training_prediction?hl=zh-tw

步驟數：7

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [設定環境](#)
4. [將訓練程式碼容器化](#)
5. [在 Vertex AI 執行訓練工作](#)
6. [部署模型端點](#)
7. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 訓練及提供 TensorFlow 模型，並在自訂容器中使用程式碼。

雖然我們在此使用 TensorFlow 做為模型程式碼，但您可以輕鬆換成其他架構。

課程內容

內容如下：

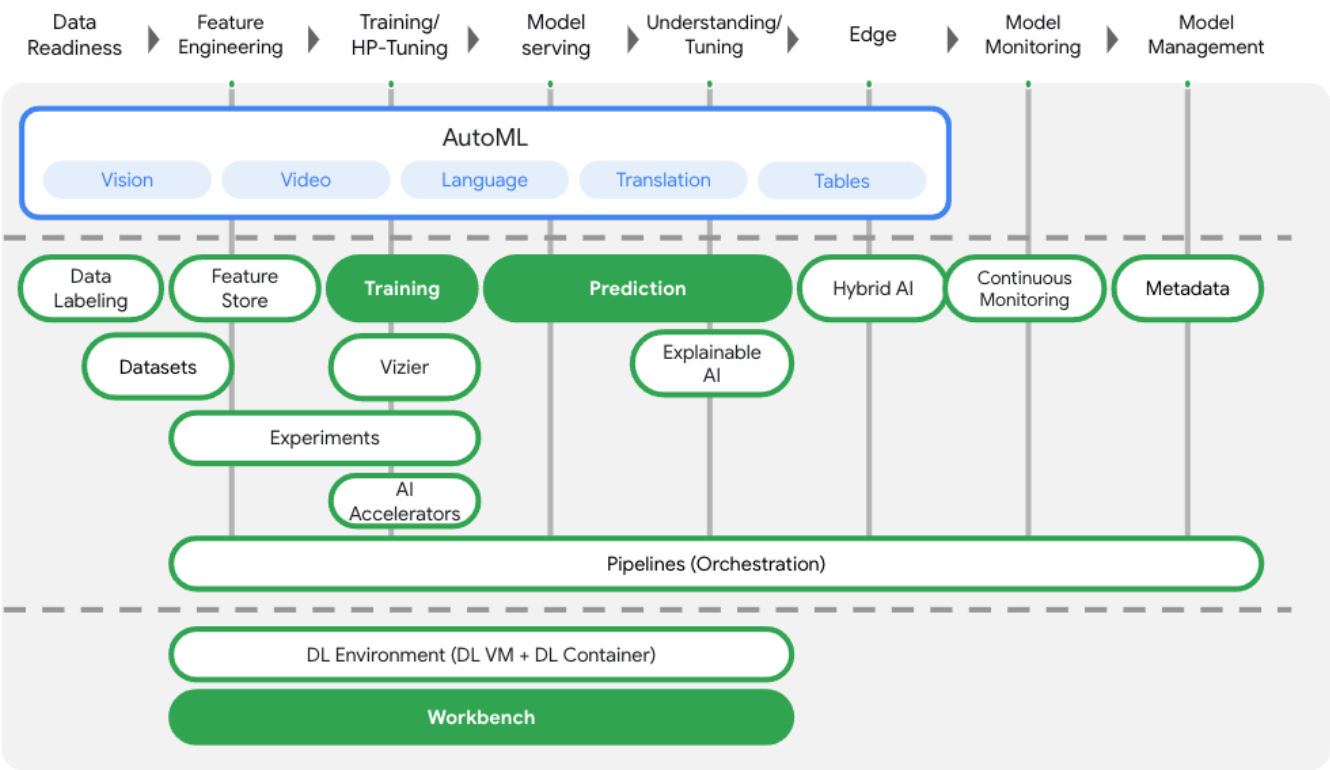
- 在 Vertex Workbench 中建構模型訓練程式碼並進行容器化
- 將自訂模型訓練工作提交至 Vertex AI
- 將訓練完的模型部署至端點，並使用該端點取得預測結果

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱[支援頁面](#)。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：訓練、預測和 Workbench。



步驟 3 · 約 10 分鐘

3. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。您需要這項資訊才能建立筆記本執行個體。

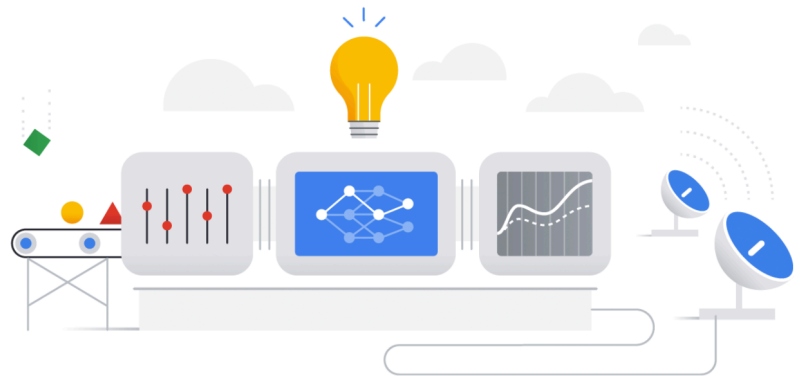
步驟 2：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 3：啟用 Container Registry API

前往「Container Registry」，然後選取「啟用」（如果尚未啟用）。您將使用這個工具為自訂訓練工作建立容器。

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



Vertex AI



Dashboard



Datasets



Features



Labelling tasks



Workbench



Pipelines

然後在「使用者自行管理的 Notebooks」中，按一下「New Notebook」(新增 Notebook)：

Notebooks	+ NEW NOTEBOOK	REFRESH	▶ START	■ STOP	🔄 RESET	🗑 DELETE
MANAGED NOTEBOOKS	PREVIEW	USER-MANAGED NOTEBOOKS	EXECUTIONS	PREVIEW	SCHEDULES	PREVIEW

然後選取最新版本的「TensorFlow 企業版 (含 LTS)」執行個體類型，且「不加入任何 GPU」：

NEW NOTEBOOK

REFRESH

START

STOP

RESET

DELETE

Customize...

Python 3

Includes scikit-learn, pandas and more

Python 3 (CUDA Toolkit 11.0)

Optimized for NVIDIA GPUs

TensorFlow Enterprise

Includes Keras, scikit-learn, pandas, NLTK and more

PyTorch 1.9

Includes scikit-learn, pandas, NLTK and more

R 4.1

Includes basic R packages, scikit-learn, pandas, NLTK and more

RAPIDS 0.18 [EXPERIMENTAL]

SCHEDULES

PREVIEW

ks instances in Workbench under the User-managed notebooks tab.

able auto-upgrade (if enabled) and

TensorFlow Enterprise 1.15 (with LTS)

TensorFlow Enterprise 2.1 (with LTS)

TensorFlow Enterprise 2.3 (with LTS)

TensorFlow Enterprise 2.6 (with LTS)

TensorFlow Enterprise 2.7

Without GPUs

With 1 NVIDIA Tesla T4

使用預設選項，然後按一下「建立」。

我們將在本實驗室中訓練及提供服務的模型，是以 TensorFlow 文件中的[這項教學課程](#)為基礎。本教學課程使用 Kaggle 的 [Auto MPG 資料集](#)，預測車輛的燃油效率。

步驟 4 · 約 5 分鐘

4. 將訓練程式碼容器化

我們會將訓練程式碼放入 [Docker 容器](#)，並將這個容器推送至 [Google Container Registry](#)，藉此將訓練工作提交至 Vertex。使用這種方法，我們可以訓練以任何架構建構的模型。

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



Notebook



Python 3



Python 2



Console



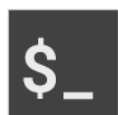
Python 3



Python 2



Other



Terminal



Text File



Markdown File



Show
Contextual Help

建立名為 `mpg` 的新目錄，然後切換至該目錄：

```
mkdir mpg
cd mpg
```

步驟 1：建立 Dockerfile

將程式碼容器化的第一步是建立 Dockerfile。在 Dockerfile 中，我們會納入執行映像檔所需的所有指令。這會安裝我們使用的所有程式庫，並設定訓練程式碼的進入點。在終端機中建立空白的 Dockerfile：

```
touch Dockerfile
```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-cpu.2-6
WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.train"]
```

這個 Dockerfile 使用 [深度學習容器 TensorFlow 企業版 2.3 Docker 映像檔](#)。Google Cloud 上的深度學習容器預先安裝了許多常見的機器學習和資料科學框架。我們使用的版本包含 TF Enterprise 2.3、Pandas、Scikit-learn 等。下載該映像檔後，這個 Dockerfile 會設定訓練程式碼的進入點。我們尚未建立這些檔案，下一個步驟會新增程式碼，用於訓練及匯出模型。

步驟 2：建立 Cloud Storage 值區

在訓練工作中，我們會將訓練後的 TensorFlow 模型匯出至 Cloud Storage 值區。Vertex 會使用這個檔案讀取匯出的模型資產，並部署模型。在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

接著，在終端機中執行下列指令，在專案中建立新的 bucket。`-l` (位置) 旗標非常重要，因為這必須與您在本教學課程稍後部署模型端點的區域相同：

```
BUCKET_NAME="gs://${PROJECT_ID}-bucket"
gsutil mb -l us-central1 $BUCKET_NAME
```

步驟 3：新增模型訓練程式碼

在終端機中執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案：

```
mkdir trainer
touch trainer/train.py
```

mpg/ 目錄現在應包含下列項目：

```
+ Dockerfile
+ trainer/
  + train.py
```

接著，開啟剛才建立的 `train.py` 檔案，然後複製下列程式碼 (改編自 TensorFlow 文件中的[教學課程](#))。

在檔案開頭，使用您在上一步建立的 Storage Bucket 名稱更新 `BUCKET` 變數：

```

import numpy as np
import pandas as pd
import pathlib
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers

print(tf.__version__)

"""## The Auto MPG dataset

The dataset is available from the [UCI Machine Learning Repository]
(https://archive.ics.uci.edu/ml/).

### Get the data
First download the dataset.
"""

dataset_path = keras.utils.get_file("auto-mpg.data", "http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data")
dataset_path

"""Import it using pandas"""

column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower', 'Weight',
                'Acceleration', 'Model Year', 'Origin']
dataset = pd.read_csv(dataset_path, names=column_names,
                      na_values = "?", comment='\t',
                      sep=" ", skipinitialspace=True)

dataset.tail()

# TODO: replace `your-gcs-bucket` with the name of the Storage bucket you created earlier
BUCKET = 'gs://your-gcs-bucket'

"""### Clean the data

The dataset contains a few unknown values.
"""

dataset.isna().sum()

"""To keep this initial tutorial simple drop those rows."""

dataset = dataset.dropna()

"""The `Origin` column is really categorical, not numeric. So convert that to a one-hot:"""

dataset['Origin'] = dataset['Origin'].map({1: 'USA', 2: 'Europe', 3: 'Japan'})

dataset = pd.get_dummies(dataset, prefix='', prefix_sep='')

```

```
dataset.tail()
```

"""### Split the data into train and test

Now split the dataset into a training set and a test set.

We will use the test set in the final evaluation of our model.

"""

```
train_dataset = dataset.sample(frac=0.8,random_state=0)
test_dataset = dataset.drop(train_dataset.index)
```

"""### Inspect the data

Have a quick look at the joint distribution of a few pairs of columns from the training set.

Also look at the overall statistics:

"""

```
train_stats = train_dataset.describe()
train_stats.pop("MPG")
train_stats = train_stats.transpose()
train_stats
```

"""### Split features from labels

Separate the target value, or "label", from the features. This label is the value that you will train the model to predict.

"""

```
train_labels = train_dataset.pop('MPG')
test_labels = test_dataset.pop('MPG')
```

"""### Normalize the data

Look again at the `train\_stats` block above and note how different the ranges of each feature are.

It is good practice to normalize features that use different scales and ranges. Although the model *might* converge without feature normalization, it makes training more difficult, and it makes the resulting model dependent on the choice of units used in the input.

Note: Although we intentionally generate these statistics from only the training dataset, these statistics will also be used to normalize the test dataset. We need to do that to project the test dataset into the same distribution that the model has been trained on.

"""

```
def norm(x):
    return (x - train_stats['mean']) / train_stats['std']
normed_train_data = norm(train_dataset)
normed_test_data = norm(test_dataset)
```

```
"""This normalized data is what we will use to train the model.
```

Caution: The statistics used to normalize the inputs here (mean and standard deviation) need to be applied to any other data that is fed to the model, along with the one-hot encoding that we did earlier. That includes the test set as well as live data when the model is used in production.

```
## The model
```

```
### Build the model
```

Let's build our model. Here, we'll use a `Sequential` model with two densely connected hidden layers, and an output layer that returns a single, continuous value. The model building steps are wrapped in a function, `build\_model`, since we'll create a second model, later on.

```
"""
```

```
def build_model():
    model = keras.Sequential([
        layers.Dense(64, activation='relu', input_shape=[len(train_dataset.keys())]),
        layers.Dense(64, activation='relu'),
        layers.Dense(1)
    ])

    optimizer = tf.keras.optimizers.RMSprop(0.001)

    model.compile(loss='mse',
                  optimizer=optimizer,
                  metrics=['mae', 'mse'])
    return model
```

```
model = build_model()
```

```
"""### Inspect the model
```

Use the `.summary` method to print a simple description of the model

```
"""
```

```
model.summary()
```

```
"""Now try out the model. Take a batch of `10` examples from the training data and call
`model.predict` on it.
```

It seems to be working, and it produces a result of the expected shape and type.

```
### Train the model
```

Train the model for 1000 epochs, and record the training and validation accuracy in the `history` object.

Visualize the model's training progress using the stats stored in the `history` object.

This graph shows little improvement, or even degradation in the validation error after about

100 epochs. Let's update the `model.fit` call to automatically stop training when the validation score doesn't improve. We'll use an `*EarlyStopping` callback* that tests a training condition for every epoch. If a set amount of epochs elapses without showing improvement, then automatically stop the training.

You can learn more about this callback [here]
(https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping).

```
model = build_model()

EPOCHS = 1000

# The patience parameter is the amount of epochs to check for improvement
early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10)

early_history = model.fit(normed_train_data, train_labels,
                          epochs=EPOCHS, validation_split = 0.2,
                          callbacks=[early_stop])

# Export model and save to GCS
model.save(BUCKET + '/mpg/model')
```

步驟 4：在本機建構及測試容器

在終端機中，使用 Google Container Registry 中的容器映像檔 URI 定義變數：

```
IMAGE_URI="gcr.io/$PROJECT_ID/mpg:v1"
```

接著，從 `mpg` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

在筆記本執行個體中執行容器，確保容器正常運作：

```
docker run $IMAGE_URI
```

模型應會在 1 到 2 分鐘內完成訓練，驗證準確率約為 72% (實際準確率可能有所不同)。在本機執行容器後，請將容器推送至 Google Container Registry：

```
docker push $IMAGE_URI
```

容器已推送至 Container Registry，現在可以開始執行自訂模型訓練工作。

步驟 5 · 約 10 分鐘

5. 在 Vertex AI 執行訓練工作

Vertex AI 提供兩種模型訓練選項：

- **AutoML**：輕鬆訓練高品質模型，僅需最基本的機器學習專業知識。
- **自訂訓練**：使用 Google Cloud 其中一個預先建構的容器，或使用自己的容器，在雲端執行自訂訓練應用程式。

在本實驗室中，我們將透過 Google Container Registry 上的自訂容器，使用自訂訓練。首先，請前往 Cloud 控制台的 Vertex 專區，然後前往「Models」專區：



Vertex AI



Dashboard



Datasets



Features



Labelling tasks



Workbench



Pipelines



Training



Experiments



Models



Endpoints



Batch predictions



Metadata

步驟 1：啟動訓練工作

按一下「建立」，輸入訓練工作和已部署模型的參數：

- 在「Dataset」(資料集) 下方，選取「No managed dataset」(沒有代管資料集)
- 然後選取「Custom training (advanced)」(自訂訓練 (進階)) 做為訓練方法，並點按「Continue」(繼續)。
- 按一下 [Continue] (繼續)。

在下一個步驟中，輸入 `mpg` (或您想為模型命名的任何名稱) 做為「模型名稱」。然後選取「自訂容器」：

Select a pre-built container or build a custom container using ML frameworks (as well as non-ML dependencies, libraries and binaries) that are not otherwise supported.

[Learn more](#)



Pre-built container

View the list of [supported runtimes](#) including TensorFlow and scikit-learn versions



Custom container

Build a custom Docker container. Must be stored in [Container Registry](#)

在「容器映像檔」文字方塊中，按一下「瀏覽」，然後找出您剛上傳至 Container Registry 的 Docker 映像檔。其餘欄位保留空白，然後點選「繼續」。

本教學課程不會使用超參數調整功能，因此請取消勾選「啟用超參數調整」方塊，然後按一下「繼續」。

在「Compute and pricing」(運算和價格) 中，保留選取的區域，然後選擇「n1-standard-4」做為機型：

Filter Type to filter

Standard	n1-standard-4 4 vCPUs, 15 GiB memory
High-memory	
High-CPU	n1-standard-8 8 vCPUs, 30 GiB memory
Custom	
High-GPU	n1-standard-16 16 vCPUs, 60 GiB memory
Mega-GPU	n1-standard-32 32 vCPUs, 120 GiB memory

CLEAR SELECTION

將加速器欄位留空，然後選取「繼續」。由於這個示範中的模型訓練速度很快，因此我們使用較小的機型。

注意：如要進行實驗，歡迎使用較大的機型和 GPU。如果您使用 GPU，則必須使用支援 GPU 的基本容器映像檔。

在「預測容器」步驟中，選取「預先建構的容器」，然後選取 TensorFlow 2.6。

保留預建容器的預設設定。在「模型目錄」下方，輸入含有 mpg 子目錄的 GCS 值區。這是模型訓練指令碼中的路徑，您會在此匯出訓練好的模型：

You can associate your custom-trained model with a container in order to serve prediction requests using Vertex AI. [Learn more about getting predictions.](#)

- ☐ No prediction container
You can always import your model artifact later to serve prediction requests
- ☒ Pre-built container
View the list of [supported runtimes](#) including TensorFlow, scikit-learn and PyTorch versions
- ☐ Custom container
Build a custom Docker container. Must be stored in [Container Registry or Artifact Registry](#)

Pre-built container settings

Vertex AI provides Docker container images for serving predictions. To use a pre-built container, your trained model code must be in Python 3.7. [Learn more about pre-built containers](#)

In order to run in a pre-built container, your code needs to be in Python 3.7

Model framework *
TensorFlow

Model framework version *
2.6

Accelerator type *
None

Model directory *
 gs:// sara-vertex-demos-bucket/mpg **BROWSE**

Cloud Storage location containing the model artifact and any supporting files

Vertex 部署模型時，會在這個位置尋找模型。現在可以開始訓練了！按一下「開始訓練」，啟動訓練工作。在控制台的「訓練」部分，您會看到類似下方的內容：

Training pipelines are the primary model training workflow in Vertex AI. You can use training pipelines to create an AutoML-trained model or a custom-trained model. For custom-trained models, training pipelines orchestrate custom training jobs and hyperparameter tuning with additional steps like adding a data set or uploading the model to Vertex AI for prediction serving. [Learn more](#)

Region

us-central1 (Iowa)

▼ ?

[Filter](#) Enter a property name

Name	ID	Status	Job type	Model type
mpg	745839419050688512	 Training	Training pipeline	 Custom

訓練工作大約需要 10 到 15 分鐘才能完成。

步驟 6 · 約 10 分鐘

6. 部署模型端點

設定訓練工作時，我們指定了 Vertex AI 應從何處尋找匯出的模型資產。在訓練管道中，Vertex 會根據這個資產路徑建立模型資源。模型資源本身並非已部署的模型，但只要有模型，您就能將其部署至端點。如要進一步瞭解 Vertex AI 中的模型和端點，請參閱[說明文件](#)。

在這個步驟中，我們會為訓練好的模型建立端點。我們可以透過 Vertex AI API，使用這個端點取得模型的預測結果。

步驟 1：部署端點

訓練工作完成後，您應該會在控制台的「Models」(模型) 區段中，看到名為「mpg」的模型 (或您為模型命名的名稱)：

Models

+

CREATE

↓

IMPORT

Models are built from your data sets or unmanaged data sources. There are many different types of machine learning models available on Vertex AI, depending on your use case and level of experience with machine learning. [Learn more](#)

Region

us-central1 (Iowa)

▼

?

Filter

Enter a property name

Name	ID	Status	Data	Endpoints	Region	Type
mpg	229476872809873408	✓ Ready	—	0	us-central1	Custom trained Custom training

訓練工作執行時，Vertex 會為您建立模型資源。如要使用這個模型，請部署端點。每個模型可以有多個端點。按一下模型，然後點選「Deploy to endpoint」(部署至端點)。

選取「建立新端點」並取名，例如 v1。將「存取權」保留為選取「標準」，然後按一下「繼續」。

將「流量分配」設為 100，並在「運算節點數量下限」輸入 1。在「Machine type」(機型) 下方，選取「n1-standard-2」(或任何您想要的機型)。其餘預設選項維持不變，然後點選「繼續」。我們不會為這個模型啟用監控功能，因此請按一下「Deploy」(部署)，開始部署端點。

部署端點需要 10 到 15 分鐘，部署完成後，您會收到電子郵件通知。端點部署完成後，您會看到下列內容，顯示模型資源下部署了一個端點：

Deploy your model

Endpoints are machine learning models made available for online prediction requests. Endpoints are useful for timely predictions from many users (for example, in response to an application request). You can also request batch predictions if you don't need immediate results.

DEPLOY TO ENDPOINT

Endpoint	ID	Models	Region	Last updated	API	Notification	Metadata
✓ v1	6681723365901729792	1	us-central1	Sep 21, 2020, 3:24:21 PM	Sample request		⋮

步驟 2：從部署的模型取得預測結果

我們將使用 Vertex Python API，從 Python 筆記本取得已訓練模型的預測結果。返回筆記本執行個體，然後從啟動器建立 Python 3 筆記本：



Notebook



Python 3



Python 2



Console



Python 3



Python 2



Other



Terminal



Text File



Markdown File



Show
Contextual Help

在筆記本的儲存格中執行下列指令，安裝 Vertex AI SDK：

```
!pip3 install google-cloud-aiplatform --upgrade --user
```

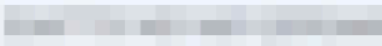
接著在筆記本中新增儲存格，匯入 SDK 並建立剛部署端點的參照：

```
from google.cloud import aiplatform

endpoint = aiplatform.Endpoint(
    endpoint_name="projects/YOUR-PROJECT-NUMBER/locations/us-central1/endpoints/YOUR-
ENDPOINT-ID"
)
```

您必須將上述 `endpoint_name` 字串中的兩個值，換成專案編號和端點。前往[專案資訊主頁](#)，即可找到專案編號值。

如要查看端點 ID，請前往控制台的端點專區：

Name	ID	Status	Models	Region
v1		 Active	1	us-central1

最後，請在新儲存格中複製並執行下列程式碼，向端點進行預測：

```
test_mpg = [1.4838871833555929,  
            1.8659883497083019,  
            2.234620276849616,  
            1.0187816540094903,  
            -2.530890710602246,  
            -1.6046416850441676,  
            -0.4651483719733302,  
            -0.4952254087173721,  
            0.7746763768735953]  
  
response = endpoint.predict([test_mpg])  
  
print('API response: ', response)  
  
print('Predicted MPG: ', response.predictions[0][0])
```

這個範例已包含標準化值，這是模型預期的格式。

執行這個儲存格，您應該會看到每加侖約 16 英里的預測輸出內容。

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 在自訂容器中提供訓練程式碼，即可訓練模型。本範例使用 TensorFlow 模型，但您可以使用自訂容器，訓練以任何架構建構的模型。
- 使用預先建構的容器部署 TensorFlow 模型，與訓練時使用的工作流程相同。
- 建立模型端點並產生預測結果。

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 7 · 約 1 分鐘

7. 清除所用資源

如要繼續使用在本實驗室中建立的筆記本，建議您在未使用時關閉筆記本。在 Cloud 控制台的 Workbench 使用者介面中，選取筆記本，然後選取「停止」。

如要徹底刪除記事本，請按一下右上方的「刪除」按鈕。

如要刪除已部署的端點，請前往 Vertex AI 控制台的「端點」部分，按一下您建立的端點，然後選取「從端點取消部署模型」：

☒	Model	Status	Most recent alerts	Monitoring	Traffic split	Compute nodes	Type	Created ↑	
☒	mpg	✓ Ready	—	Disabled	100%	Auto (1 minimum, 1 maximum)	Custom	2 Dec 2021,	Undeploy model from endpoint

DEPLOY ANOTHER MODEL

如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

Storage browser		CREATE BUCKET	DELETE	REFRESH
	Filter buckets			
	Name	Created ↓		
☒	bucket_of_fun	Sep 16, 2020, 11:16:30 AM		